

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

Кафедра Информационных систем

Практическая работа

по дисциплине «Теория принятия решений»

Тема: "Применение методов динамического программирования для
решения задачи поиска оптимального плана заказа станков"

Студент гр. 3376

Преподаватель

Михайлов Н.Ф.

Пономарёв А.В.

Санкт-Петербург
2026

Содержание

1	Введение	2
2	Постановка задачи	2
3	Формализация задачи и вывод уравнения Беллмана	2
3.1	Этапы, состояние и управление	2
3.2	Ограничения	3
3.3	Уравнение Беллмана	3
4	Решение подзадач	4
4.1	Подзадача 1	4
4.2	Подзадача 2	4
4.3	Подзадача 3	5
4.4	Подзадача 4	5
5	Сравнение скорость работы разных реализаций	5
5.1	Результаты измерений	5
6	Заключение	6
A	Листинг кода	7
A.1	Реализация на C++	7
A.2	Реализация на Python	14

1 Введение

Целью в данной практической работе является освоение метода динамического программирования в рамках задачи о поиске оптимального плана заказа станков.

Для этого нам необходимо

1. Формализовать задачу
2. Составить уравнение Беллмана
3. Реализовать алгоритм (прямой или обратной прогонки)
4. Протестировать алгоритм и проанализировать его итоговые данные

2 Постановка задачи

Склад пункта реализации станков имеет вместимость 20 единиц. Пополнение склада возможно только первого числа каждого месяца. Станки привозят автотранспортом (1 рейс), причем стоимость рейса складывается из постоянных затрат 50 денежных единиц (ДЕ) и затрат на доставку каждого станка 10 ДЕ. За один рейс (и, следовательно, в месяц) может быть доставлено не более 5 станков.

Затраты на хранение станка в течение месяца составляют 10 ДЕ.

Ожидаемый спрос на станки приведен в табл. 1. Отсутствие требуемого количества станков на складе недопустимо. В начальный момент на складе находится 1 станков.

1. Определить план заказов, минимизирующий стоимость
2. Определить границы изменения стоимости хранения станка, в которых найденный план является оптимальным
3. Определить границы изменения постоянных затрат на совершение рейса, в которых найденный план является оптимальным
4. Определить план заказов, минимизирующий стоимость, при условии, что к концу периода на складе должно остаться 2 станков

3 Формализация задачи и вывод уравнения Беллмана

3.1 Этапы, состояние и управление

В качестве этапов рассматриваются месяцы планового периода, то есть

$$i = 1, \dots, 6.$$

Состояние системы на шаге i определяется количеством станков на складе в начале месяца. Обозначим это состояние через

$$S_i.$$

Управлением на шаге i является количество станков, заказываемых в начале месяца:

$$u_i.$$

Так как за один месяц может быть выполнен только один рейс, а за один рейс можно доставить не более 5 станков, управление ограничено:

$$0 \leq u_i \leq 5.$$

Переход системы из состояния S_i в состояние S_{i+1} задаётся уравнением баланса:

$$S_{i+1} = S_i + u_i - d_i,$$

где d_i — спрос на станки в месяце i .

Таким образом, состояние на следующем шаге зависит от текущего запаса на складе, принятого решения о заказе и спроса в рассматриваемом месяце.

3.2 Ограничения

При решении задачи необходимо учитывать следующие ограничения.

1. Вместимость склада не должна превышать:

$$0 \leq S_i \leq 20.$$

2. Дефицит станков недопустим, поэтому имеющегося запаса и объёма заказа должно хватать на покрытие спроса:

$$S_i + u_i \geq d_i.$$

3. За один месяц можно заказать не более 5 станков:

$$0 \leq u_i \leq 5.$$

4. Для подзадачи 4 добавляется дополнительное ограничение на конечный остаток:

$$S_7 = 2.$$

Эти ограничения определяют множество допустимых решений, среди которых выбирается решение с минимальной стоимостью.

3.3 Уравнение Беллмана

Пусть

$$W_i(S_i)$$

— минимальные суммарные затраты от месяца i до конца периода, если в начале месяца i на складе находится S_i станков.

Тогда уравнение Беллмана для данной задачи имеет вид:

$$W_i(S_i) = \min_{u_i \in U_i(S_i)} [c(u_i) + hS_{i+1} + W_{i+1}(S_{i+1})],$$

где:

- h — стоимость хранения одного станка;
- $S_{i+1} = S_i + u_i - d_i$;

- $U_i(S_i)$ — множество допустимых управлений;
- $c(u_i)$ — стоимость поставки в месяце i .

Стоимость поставки определяется следующим образом:

$$c(u_i) = \{ 0, u_i = 0, 50 + 10u_i, u_i > 0. \}$$

Тогда уравнение Беллмана можно записать более подробно:

$$W_i(S_i) = \min_{u_i} [\{ 0, u_i = 0, 50 + 10u_i, u_i > 0 + 10S_{i+1} + W_{i+1}(S_{i+1}) \}].$$

Граничное условие для базовой задачи:

$$W_7(S_7) = 0.$$

Для подзадачи 4, где требуется получить конечный остаток 2 станка:

$$W_7(S_7) = \{ 0, S_7 = 2, +\infty, S_7 \neq 2. \}$$

Это уравнение лежит в основе реализованного рекурсивного алгоритма.

4 Решение подзадач

4.1 Подзадача 1

В первой подзадаче требуется определить оптимальный план заказов, минимизирующий общую стоимость поставки и хранения.

Программа последовательно рассматривает все допустимые объёмы заказа в каждом месяце, вычисляет стоимость текущего решения и рекурсивно добавляет минимальную стоимость на оставшемся горизонте планирования. После нахождения минимальной стоимости восстанавливается сам план заказов по месяцам.

В результате был получен следующий оптимальный план:

$$(2, 5, 5, 5, 0, 0).$$

Минимальная суммарная стоимость:

$$450.$$

Это решение означает, что выгодно сделать поставки в первые четыре месяца, а затем использовать накопленный запас на складе.

4.2 Подзадача 2

Во второй подзадаче требуется определить границы изменения стоимости хранения одного станка, при которых найденный в первой подзадаче план остаётся оптимальным.

Для этого фиксируется базовый оптимальный план, после чего параметр стоимости хранения последовательно уменьшается и увеличивается. Для каждого нового значения параметра задача решается заново, а полученный план сравнивается с базовым.

В результате установлено, что базовый план остаётся оптимальным при стоимости хранения в диапазоне:

$$[0; 12].$$

Следовательно, если стоимость хранения не превышает 12 денежных единиц за станок в месяц, структура оптимального плана не изменяется.

4.3 Подзадача 3

В третьей подзадаче исследуется влияние постоянных затрат на совершение рейса на оптимальность базового плана.

Аналогично подзадаче 2, фиксируется найденный план и выполняется параметрическое изменение постоянной составляющей стоимости рейса. При каждом новом значении параметра задача решается заново и сравнивается с исходным решением.

Нижняя граница изменения постоянных затрат составляет:

$$41.$$

Верхняя граница отсутствует. Это объясняется тем, что найденный базовый план использует минимально возможное число рейсов. Так как при увеличении постоянной стоимости рейса все планы с большим количеством рейсов становятся только менее выгодными, базовый план остаётся оптимальным и далее.

Таким образом, диапазон имеет вид:

$$[41; +\infty).$$

4.4 Подзадача 4

В четвёртой подзадаче требуется определить оптимальный план заказов при дополнительном условии:

$$S_7 = 2.$$

Для этого в граничное условие рекурсивного алгоритма вводится требование, чтобы к концу периода на складе осталось ровно 2 станка. Все остальные конечные состояния считаются недопустимыми.

В результате получен следующий план:

$$(2, 5, 5, 3, 0, 4).$$

Минимальная стоимость при этом составляет:

$$500.$$

По сравнению с базовой задачей стоимость возрастает, поскольку необходимо обеспечить дополнительный запас к концу периода.

5 Сравнение скорости работы разных реализаций

Для оценки эффективности были реализованы несколько версий алгоритма: на C++ и Python. Дополнительно была предусмотрена возможность запуска с мемоизацией и без неё.

Сравнение выполнялось с помощью сценария автоматического тестирования, который запускал каждую реализацию в одинаковых условиях и измерял время выполнения.

5.1 Результаты измерений

Для C++ и Python были получены следующие результаты:

- C++ с мемоизацией — 0.004 с;

- C++ без мемоизации — 0.007 с;
- Python с мемоизацией — 0.017 с;
- Python без мемоизации — 0.060 с.

Из этих данных можно сделать следующие выводы:

- для реализации на C++ использование мемоизации уменьшило время работы с 0.007 с до 0.004 с, то есть примерно на 42.9%;
- для реализации на Python использование мемоизации уменьшило время работы с 0.060 с до 0.017 с, то есть примерно на 71.7%;
- при использовании мемоизации Python работает примерно в 4.25 раза медленнее C++, что соответствует увеличению времени примерно на 325% относительно C++;
- без мемоизации Python работает примерно в 8.57 раза медленнее C++, что соответствует увеличению времени примерно на 757% относительно C++.

Следует отметить, что сама по себе рассматриваемая задача не является вычислительно очень сложной, поэтому все измеренные времена остаются малыми и на практике оба варианта работают быстро. Однако в относительном выражении разница оказывается весьма существенной.

Таким образом, мемоизация существенно повышает эффективность рекурсивного алгоритма динамического программирования, а выбор языка реализации также заметно влияет на производительность программы.

6 Заключение

В ходе работы была формализована задача оптимального планирования заказов станков и построено её решение методом динамического программирования на основе уравнения Беллмана.

Был реализован алгоритм, позволяющий:

- находить оптимальный план заказов;
- исследовать устойчивость найденного решения к изменению параметров;
- учитывать дополнительное условие на конечный остаток на складе.

Проведённые вычисления показали, что метод динамического программирования позволяет эффективно решать задачу данного типа, а использование мемоизации заметно сокращает время работы алгоритма. Полученные результаты подтверждают применимость данного подхода для задач оптимального управления запасами.

А Листинг кода

А.1 Реализация на C++

```
1 #include <iostream>
2 #include <vector>
3 #include <climits>
4 #include <string>
5
6 std::vector<int> DEMAND = {0, 7, 6, 3, 0, 2};
7 #define INITIAL_NUMBER_OF_MACHINES 1
8 #define WAREHOUSE_CAPACITY 20
9 #define MAX_ORDER 5
10 #define DELIVERY_COST 50
11 #define PER_MACHINE_COST 10
12 #define STORAGE_COST 10
13
14 bool USE_MEMOIZATION = true;
15
16 int min_cost(
17     int month,
18     int warehouse,
19     const std::vector<int>& demand,
20     std::vector<std::vector<int>>& dp,
21     std::vector<std::vector<bool>>& used,
22     bool use_memoization = true,
23     int final_warehouse = -1,
24     int storage_cost = STORAGE_COST,
25     int delivery_cost = DELIVERY_COST) {
26
27     if (month == demand.size()) {
28         if (final_warehouse == -1 || warehouse == final_warehouse) {
29             return 0;
30         }
31         return INT_MAX;
32     }
33
34     if (use_memoization && used[month][warehouse]) {
35         return dp[month][warehouse];
36     }
37
38     int best = INT_MAX;
39
40     for (int order = 0; order <= MAX_ORDER; order++) {
41         if (warehouse + order < demand[month]) {
42             continue;
43         }
44
45         int next_warehouse = warehouse + order - demand[month];
46         if (next_warehouse > WAREHOUSE_CAPACITY) {
47             continue;
48         }
49     }
```



```

50     int step_cost = 0;
51     if (order > 0) {
52         step_cost += delivery_cost + PER_MACHINE_COST * order;
53     }
54     step_cost += storage_cost * next_warehouse;
55
56     int future_cost = min_cost(
57         month + 1,
58         next_warehouse,
59         demand,
60         dp,
61         used,
62         use_memoization,
63         final_warehouse,
64         storage_cost,
65         delivery_cost
66     );
67     if (future_cost == INT_MAX) {
68         continue;
69     }
70
71     int total = step_cost + future_cost;
72     if (total < best) {
73         best = total;
74     }
75 }
76
77 if (use_memoization) {
78     used[month][warehouse] = true;
79     dp[month][warehouse] = best;
80 }
81 return best;
82 }
83
84 int machines_num(
85     int month,
86     int warehouse,
87     const std::vector<int>& demand,
88     std::vector<std::vector<int>>& dp,
89     std::vector<std::vector<bool>>& used,
90     bool use_memoization = true,
91     int final_warehouse = -1,
92     int storage_cost = STORAGE_COST,
93     int delivery_cost = DELIVERY_COST) {
94
95     if (month == demand.size()) {
96         return 0;
97     }
98
99     int best = INT_MAX;
100    int best_order = 0;
101
102    for (int order = 0; order <= MAX_ORDER; order++) {

```

```

103     if (warehouse + order < demand[month]) {
104         continue;
105     }
106
107     int next_warehouse = warehouse + order - demand[month];
108     if (next_warehouse > WAREHOUSE_CAPACITY) {
109         continue;
110     }
111
112     int step_cost = 0;
113     if (order > 0) {
114         step_cost += delivery_cost + PER_MACHINE_COST * order;
115     }
116     step_cost += storage_cost * next_warehouse;
117
118     int future_cost = min_cost(
119         month + 1,
120         next_warehouse,
121         demand,
122         dp,
123         used,
124         use_memoization,
125         final_warehouse,
126         storage_cost,
127         delivery_cost
128     );
129     if (future_cost == INT_MAX) {
130         continue;
131     }
132
133     int total = step_cost + future_cost;
134     if (total < best) {
135         best = total;
136         best_order = order;
137     }
138 }
139
140 return best_order;
141 }
142
143 void minimize_cost(
144     std::vector<int>& plan,
145     int& total_cost,
146     const std::vector<int>& demand,
147     bool use_memoization = true,
148     int final_warehouse = -1,
149     int storage_cost = STORAGE_COST,
150     int delivery_cost = DELIVERY_COST) {
151     int warehouse = INITIAL_NUMBER_OF_MACHINES;
152     std::vector<std::vector<int>> dp(
153         demand.size() + 1,
154         std::vector<int>(WAREHOUSE_CAPACITY + 1, 0)
155 );

```

```

156     std::vector<std::vector<bool>> used(
157         demand.size() + 1,
158         std::vector<bool>(WAREHOUSE_CAPACITY + 1, false)
159     );
160
161     total_cost = min_cost(
162         0,
163         warehouse,
164         demand,
165         dp,
166         used,
167         use_memoization,
168         final_warehouse,
169         storage_cost,
170         delivery_cost
171     );
172     plan.clear();
173
174     for (int month = 0; month < demand.size(); month++) {
175         int order = machines_num(
176             month,
177             warehouse,
178             demand,
179             dp,
180             used,
181             use_memoization,
182             final_warehouse,
183             storage_cost,
184             delivery_cost
185         );
186         plan.push_back(order);
187         warehouse = warehouse + order - demand[month];
188     }
189 }
190
191 std::vector<int> task1(int final_warehouse = -1) {
192     std::vector<int> plan;
193     int total_cost = 0;
194
195     minimize_cost(plan, total_cost, DEMAND, USE_MEMOIZATION, final_warehouse);
196
197     std::cout << "Optimal order plan "
198         << (final_warehouse == -1 ? "without constraint" : "with
199 constraint")
200         << " on final warehouse stock";
201
202     if (final_warehouse != -1) {
203         std::cout << " (" << final_warehouse << ")";
204     }
205
206     std::cout << ": ";
207
208     for (int order : plan) {

```

```

208         std::cout << order << " ";
209     }
210     std::cout << "\nTotal cost: " << total_cost << std::endl;
211
212     return plan;
213 }
214
215 int task2(std::vector<int> base_plan = {}) {
216     int base_cost = 0;
217     if (base_plan.empty()) {
218         minimize_cost(base_plan, base_cost, DEMAND, USE_MEMOIZATION, -1,
219 STORAGE_COST);
220     }
221
222     int left = STORAGE_COST;
223     int right = STORAGE_COST;
224
225     for (int cost = STORAGE_COST - 1; cost >= 0; cost--) {
226         std::vector<int> current_plan;
227         int current_cost = 0;
228         minimize_cost(current_plan, current_cost, DEMAND, USE_MEMOIZATION, -1,
229 cost);
230
231         if (current_plan == base_plan) {
232             left = cost;
233         } else {
234             break;
235         }
236     }
237
238     for (int cost = STORAGE_COST + 1; ; cost++) {
239         std::vector<int> current_plan;
240         int current_cost = 0;
241         minimize_cost(current_plan, current_cost, DEMAND, USE_MEMOIZATION, -1,
242 cost);
243
244         if (current_plan == base_plan) {
245             right = cost;
246         } else {
247             break;
248         }
249     }
250
251     std::cout << "Storage cost range: ["
252         << left << ", " << right << "]" << std::endl;
253
254     return 0;
255 }
256
257 int task3(std::vector<int> base_plan = {}) {
258     int base_cost = 0;
259     if (base_plan.empty()) {

```

```

257     minimize_cost(base_plan, base_cost, DEMAND, USE_MEMOIZATION, -1,
258 STORAGE_COST, DELIVERY_COST);
259 }
260
261 int left = DELIVERY_COST;
262 int right = DELIVERY_COST;
263
264 int total_demand = 0;
265 for (int value : DEMAND) {
266     total_demand += value;
267 }
268
269 int required = total_demand - INITIAL_NUMBER_OF_MACHINES;
270 int min_trips = (required + MAX_ORDER - 1) / MAX_ORDER;
271
272 int base_trips = 0;
273 for (int order : base_plan) {
274     if (order > 0) {
275         base_trips++;
276     }
277 }
278
279 for (int cost = DELIVERY_COST - 1; cost >= 0; cost--) {
280     std::vector<int> current_plan;
281     int current_cost = 0;
282     minimize_cost(current_plan, current_cost, DEMAND, USE_MEMOIZATION, -1,
283 STORAGE_COST, cost);
284
285     if (current_plan == base_plan) {
286         left = cost;
287     } else {
288         break;
289     }
290 }
291
292 for (int cost = DELIVERY_COST + 1; ; cost++) {
293     if (base_trips == min_trips) {
294         std::cout << "No upper bound: the base plan uses "
295 << min_trips
296 << " trips, which is the minimum possible number of trips.
297 "
298 << std::endl;
299     }
300     right = -1;
301     break;
302
303     std::vector<int> current_plan;
304     int current_cost = 0;
305     minimize_cost(current_plan, current_cost, DEMAND, USE_MEMOIZATION, -1,
306 STORAGE_COST, cost);
307
308     if (current_plan == base_plan) {
309         right = cost;

```

```

306         } else {
307             break;
308         }
309     }
310
311     std::cout << "Trip fixed cost range: ["
312         << left << ", " << right << "]" << std::endl;
313
314     return 0;
315 }
316
317 void task4(int final_warehouse) {
318     auto plan = task1(final_warehouse);
319     std::vector<int> remains = {};
320     for (int i = 0; i < plan.size(); ++i) {
321         remains.push_back(plan[i] - DEMAND[i]);
322     }
323     std::cout << "Stock at the beginning of each month: ";
324     for (int remain : remains) {
325         std::cout << remain << " ";
326     }
327     std::cout << std::endl;
328 }
329
330 int main(int argc, char* argv[]) {
331     for (int i = 1; i < argc; i++) {
332         std::string arg = argv[i];
333         if (arg == "--memo") {
334             USE_MEMOIZATION = true;
335         } else if (arg == "--no-memo") {
336             USE_MEMOIZATION = false;
337         } else {
338             std::cout << "Unknown argument: " << arg << std::endl;
339             std::cout << "Usage: ./lab1 [--memo | --no-memo]" << std::endl;
340             return 1;
341         }
342     }
343
344     std::cout << "\n===== Task 1 =====\n";
345     auto plan = task1();
346
347     std::cout << "\n===== Task 2 =====\n";
348     task2(plan);
349
350     std::cout << "\n===== Task 3 =====\n";
351     task3(plan);
352
353     std::cout << "\n===== Task 4 =====\n";
354     task4(2);
355     return 0;
356 }

```

A.2 Реализация на Python

```
1 from __future__ import annotations
2
3 import sys
4
5 DEMAND = [0, 7, 6, 3, 0, 2]
6 INITIAL_NUMBER_OF_MACHINES = 1
7 WAREHOUSE_CAPACITY = 20
8 MAX_ORDER = 5
9 DELIVERY_COST = 50
10 PER_MACHINE_COST = 10
11 STORAGE_COST = 10
12
13 USE_MEMOIZATION = True
14
15
16 def min_cost(
17     month: int,
18     warehouse: int,
19     demand: list[int],
20     dp: list[list[int]],
21     used: list[list[bool]],
22     use_memoization: bool = True,
23     final_warehouse: int = -1,
24     storage_cost: int = STORAGE_COST,
25     delivery_cost: int = DELIVERY_COST,
26 ) -> int:
27     if month == len(demand):
28         if final_warehouse == -1 or warehouse == final_warehouse:
29             return 0
30         return sys.maxsize
31
32     if use_memoization and used[month][warehouse]:
33         return dp[month][warehouse]
34
35     best = sys.maxsize
36
37     for order in range(MAX_ORDER + 1):
38         if warehouse + order < demand[month]:
39             continue
40
41         next_warehouse = warehouse + order - demand[month]
42         if next_warehouse > WAREHOUSE_CAPACITY:
43             continue
44
45         step_cost = 0
46         if order > 0:
47             step_cost += delivery_cost + PER_MACHINE_COST * order
48         step_cost += storage_cost * next_warehouse
49
50         future_cost = min_cost(
51             month + 1,
```

```

52         next_warehouse,
53         demand,
54         dp,
55         used,
56         use_memoization,
57         final_warehouse,
58         storage_cost,
59         delivery_cost,
60     )
61     if future_cost == sys.maxsize:
62         continue
63
64     total = step_cost + future_cost
65     if total < best:
66         best = total
67
68     if use_memoization:
69         used[month][warehouse] = True
70         dp[month][warehouse] = best
71
72     return best
73
74
75 def machines_num(
76     month: int,
77     warehouse: int,
78     demand: list[int],
79     dp: list[list[int]],
80     used: list[list[bool]],
81     use_memoization: bool = True,
82     final_warehouse: int = -1,
83     storage_cost: int = STORAGE_COST,
84     delivery_cost: int = DELIVERY_COST,
85 ) -> int:
86     if month == len(demand):
87         return 0
88
89     best = sys.maxsize
90     best_order = 0
91
92     for order in range(MAX_ORDER + 1):
93         if warehouse + order < demand[month]:
94             continue
95
96         next_warehouse = warehouse + order - demand[month]
97         if next_warehouse > WAREHOUSE_CAPACITY:
98             continue
99
100         step_cost = 0
101         if order > 0:
102             step_cost += delivery_cost + PER_MACHINE_COST * order
103         step_cost += storage_cost * next_warehouse
104

```



```

105         future_cost = min_cost(
106             month + 1,
107             next_warehouse,
108             demand,
109             dp,
110             used,
111             use_memoization,
112             final_warehouse,
113             storage_cost,
114             delivery_cost,
115         )
116         if future_cost == sys.maxsize:
117             continue
118
119         total = step_cost + future_cost
120         if total < best:
121             best = total
122             best_order = order
123
124     return best_order
125
126
127 def minimize_cost(
128     plan: list[int],
129     demand: list[int],
130     use_memoization: bool = True,
131     final_warehouse: int = -1,
132     storage_cost: int = STORAGE_COST,
133     delivery_cost: int = DELIVERY_COST,
134 ) -> int:
135     warehouse = INITIAL_NUMBER_OF_MACHINES
136     dp = [[0] * (WAREHOUSE_CAPACITY + 1) for _ in range(len(demand) + 1)]
137     used = [[False] * (WAREHOUSE_CAPACITY + 1) for _ in range(len(demand) + 1)]
138
139     total_cost = min_cost(
140         0,
141         warehouse,
142         demand,
143         dp,
144         used,
145         use_memoization,
146         final_warehouse,
147         storage_cost,
148         delivery_cost,
149     )
150     plan.clear()
151
152     for month in range(len(demand)):
153         order = machines_num(
154             month,
155             warehouse,
156             demand,
157             dp,

```

```

158         used,
159         use_memoization,
160         final_warehouse,
161         storage_cost,
162         delivery_cost,
163     )
164     plan.append(order)
165     warehouse = warehouse + order - demand[month]
166
167     return total_cost
168
169
170 def task1(final_warehouse: int = -1) -> list[int]:
171     plan: list[int] = []
172     total_cost = minimize_cost(plan, DEMAND, USE_MEMOIZATION, final_warehouse)
173
174     print(
175         "Optimal order plan "
176         + ("without constraint" if final_warehouse == -1 else "with constraint")
177         + " on final warehouse stock",
178         end="",
179     )
180
181     if final_warehouse != -1:
182         print(f" ({{final_warehouse}})", end="")
183
184     print(": ", end="")
185
186     for order in plan:
187         print(order, end=" ")
188     print(f"\nTotal cost: {{total_cost}}")
189
190     return plan
191
192
193 def task2(base_plan: list[int] | None = None) -> int:
194     if base_plan is None:
195         base_plan = []
196
197     if not base_plan:
198         minimize_cost(base_plan, DEMAND, USE_MEMOIZATION, -1, STORAGE_COST)
199
200     left = STORAGE_COST
201     right = STORAGE_COST
202
203     for cost in range(STORAGE_COST - 1, -1, -1):
204         current_plan: list[int] = []
205         minimize_cost(current_plan, DEMAND, USE_MEMOIZATION, -1, cost)
206
207         if current_plan == base_plan:
208             left = cost
209         else:
210             break

```

```

211
212     cost = STORAGE_COST + 1
213     while True:
214         current_plan = []
215         minimize_cost(current_plan, DEMAND, USE_MEMOIZATION, -1, cost)
216
217         if current_plan == base_plan:
218             right = cost
219             cost += 1
220         else:
221             break
222
223     print(f"Storage cost range: [{left}, {right}]")
224     return 0
225
226
227 def task3(base_plan: list[int] | None = None) -> int:
228     if base_plan is None:
229         base_plan = []
230
231     if not base_plan:
232         minimize_cost(
233             base_plan,
234             DEMAND,
235             USE_MEMOIZATION,
236             -1,
237             STORAGE_COST,
238             DELIVERY_COST,
239         )
240
241     left = DELIVERY_COST
242     right = DELIVERY_COST
243
244     total_demand = 0
245     for value in DEMAND:
246         total_demand += value
247
248     required = total_demand - INITIAL_NUMBER_OF_MACHINES
249     min_trips = (required + MAX_ORDER - 1) // MAX_ORDER
250
251     base_trips = 0
252     for order in base_plan:
253         if order > 0:
254             base_trips += 1
255
256     for cost in range(DELIVERY_COST - 1, -1, -1):
257         current_plan: list[int] = []
258         minimize_cost(current_plan, DEMAND, USE_MEMOIZATION, -1, STORAGE_COST,
259                        cost)
260
261         if current_plan == base_plan:
262             left = cost
263     else:

```

```

263         break
264
265     cost = DELIVERY_COST + 1
266     while True:
267         if base_trips == min_trips:
268             print(
269                 "No upper bound: the base plan uses "
270                 f"{min_trips} trips, which is the minimum possible number of
trips."
271             )
272             right = -1
273             break
274
275     current_plan = []
276     minimize_cost(current_plan, DEMAND, USE_MEMOIZATION, -1, STORAGE_COST,
cost)
277
278     if current_plan == base_plan:
279         right = cost
280         cost += 1
281     else:
282         break
283
284     print(f"Trip fixed cost range: [{left}, {right}]")
285     return 0
286
287
288 def task4(final_warehouse: int) -> None:
289     plan = task1(final_warehouse)
290     remains: list[int] = []
291     for i in range(len(plan)):
292         remains.append(plan[i] - DEMAND[i])
293
294     print("Stock at the beginning of each month: ", end="")
295     for remain in remains:
296         print(remain, end=" ")
297     print()
298
299
300 def main() -> int:
301     global USE_MEMOIZATION
302
303     for arg in sys.argv[1:]:
304         if arg == "--memo":
305             USE_MEMOIZATION = True
306         elif arg == "--no-memo":
307             USE_MEMOIZATION = False
308         else:
309             print(f"Unknown argument: {arg}")
310             print("Usage: ./lab1.py [--memo | --no-memo]")
311             return 1
312
313     print("\n===== Task 1 =====")

```

```
314     plan = task1()
315
316     print("\n===== Task 2 =====")
317     task2(plan)
318
319     print("\n===== Task 3 =====")
320     task3(plan)
321
322     print("\n===== Task 4 =====")
323     task4(2)
324     return 0
325
326
327 if __name__ == "__main__":
328     raise SystemExit(main())
```